Instituto Tecnológico de Costa Rica
Centro Académico de Alajuela
Ingeniería en Computación
IC-2101. Programación Orientada a Objetos
Prof. Emmanuel Ramírez Segura
Semestre I - 2026
Valor: 20%
Fecha de asignación: 09/04/2026
Fecha de entrega: 12/05/2026

Primer Proyecto Programado:
Implementación del juego de Ajedrez utilizando POO en Java

➢ Objetivo

Desarrollar un sistema de ajedrez en Java que permita gestionar partidas entre jugadores, controlar las reglas del juego, registrar los movimientos realizados y administrar el estado completo de cada partida.

Para cumplir con este objetivo, el grupo desarrollador deberá construir una aplicación robusta que posibilite a dos jugadores disputar una partida de ajedrez, aplicando los principios de la Programación Orientada a Objetos (POO) y ofreciendo interacción mediante una interfaz gráfica intuitiva.

➢ Alcance del Sistema

El sistema deberá representar de manera fiel la lógica del ajedrez, incluyendo la modelación de las piezas, el tablero, la gestión de turnos, la validación de movimientos, la captura de piezas y la detección de situaciones especiales como jaque, jaque mate y promociones.

Como parte del proceso de desarrollo, se requiere cumplir con las siguientes etapas:

En primer lugar, el sistema deberá ser modelado mediante un Diagrama de Clases UML, en el cual se identifiquen de manera correcta las clases involucradas, sus atributos, métodos, cardinalidades y las relaciones existentes entre ellas.

Posteriormente, se deberá implementar el sistema en Java, respetando fielmente el diseño propuesto en el diagrama UML y cumpliendo los requerimientos funcionales que se detallan más adelante.

➢ Contenidos del curso a poner en práctica

Modelado UML de Clases, Conceptos de POO (Herencia, Asociaciones, Polimorfismo, Reutilización, Encapsulamiento, Abstracción, Clases Concretas, Clases Abstractas, Constructores, Métodos Sets y Gets, Métodos personalizados para validaciones, entre otros).

➢ **Descripción del funcionamiento**

El sistema del Ajedrez estará compuesto por varias clases principales que modelan los elementos fundamentales de una partida de ajedrez. Entre estas clases se encuentran Jugador, Tablero, Casilla, Pieza y las subclases concretas de cada tipo de pieza.

La clase Jugador representa a cada participante de la partida. El sistema deberá crear exactamente dos objetos de esta clase: un jugador con color blanco y un jugador con color negro. Cada jugador tendrá como atributos su nombre, su color asignado y su estado dentro de la partida. El estado podrá indicar si el jugador se encuentra en juego, si ha resultado ganador o si ha resultado perdedor. La clase Jugador se relaciona con las piezas, ya que cada jugador controla únicamente las piezas de su color.

La clase Tablero representa el espacio de juego. Su principal atributo será una estructura de tipo matriz de 8 filas por 8 columnas, la cual almacenará las posiciones del tablero. El tablero será responsable de inicializar las piezas en su posición estándar al inicio de la partida, verificar si una casilla está ocupada o vacía y actualizar las posiciones cuando se realice un movimiento. El tablero se relaciona con la clase Casilla, ya que está compuesto por 64 casillas organizadas en forma matricial.

La clase Casilla representa cada una de las posiciones individuales del tablero. Cada casilla tendrá como atributos una fila, una columna y una referencia a una pieza. Si la casilla no contiene ninguna pieza, esta referencia podrá ser nula. La relación entre Tablero y Casilla es de composición, ya que las casillas existen como parte del tablero y no tendrían sentido de manera independiente dentro del sistema.

La clase Pieza debe modelarse como una clase general o abstracta, ya que representa las características comunes de todas las piezas del ajedrez. Esta clase tendrá como atributos el color, la posición actual y el estado de la pieza, indicando si está activa o capturada. Además, la clase Pieza deberá definir el comportamiento general que todas las piezas comparten, especialmente la validación de movimientos. A partir de esta clase se derivarán las subclases concretas Rey, Reina, Torre, Alfil, Caballo y Peón.

La relación entre Pieza y sus tipos específicos es una relación de herencia, ya que todas las piezas comparten atributos comunes, pero cada una implementa una lógica de movimiento particular. Por ejemplo, la clase Rey validará movimientos de una casilla en cualquier dirección, la Reina combinará movimientos de torre y alfil, la Torre se moverá en líneas rectas horizontales o verticales, el Alfil en diagonales, el Caballo en forma de L y el Peón con sus restricciones especiales de avance y captura.

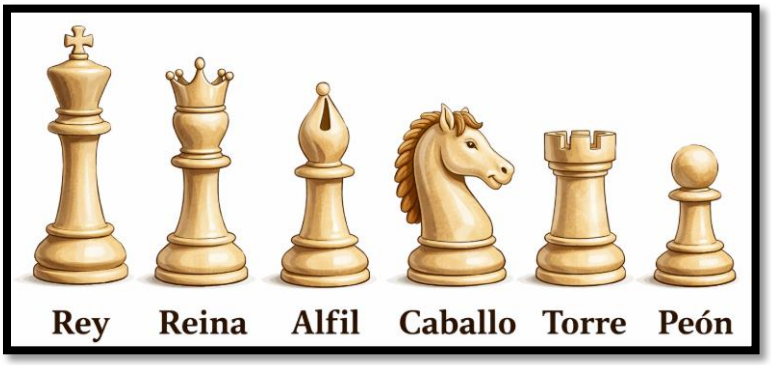
Cada objeto de tipo Pieza deberá estar ubicado en una Casilla del tablero. Por lo tanto, existe una asociación entre Casilla y Pieza: una casilla puede contener cero o una pieza, mientras que una pieza activa ocupa exactamente una casilla del tablero.

Adicionalmente, el sistema puede incluir una clase Movimiento, cuya función será registrar cada acción realizada durante la partida. Esta clase podrá contener como atributos la pieza movida, la posición de origen, la posición de destino y el tipo de movimiento realizado. La clase Movimiento se relaciona con Pieza y con las posiciones involucradas en el desplazamiento.

Finalmente, el sistema puede ser administrado por una clase Partida, encargada de coordinar todo el juego. Esta clase tendría la responsabilidad de mantener las referencias a los dos jugadores, al tablero, al turno actual, al estado general de la partida y al historial de movimientos realizados. La clase Partida sería la encargada de controlar el flujo general del juego, verificar alternancia de turnos, detectar jaque, jaque mate o promoción, y finalizar la partida cuando corresponda.

➢ Detalles importantes

- *Referencia gráfica de las piezas*



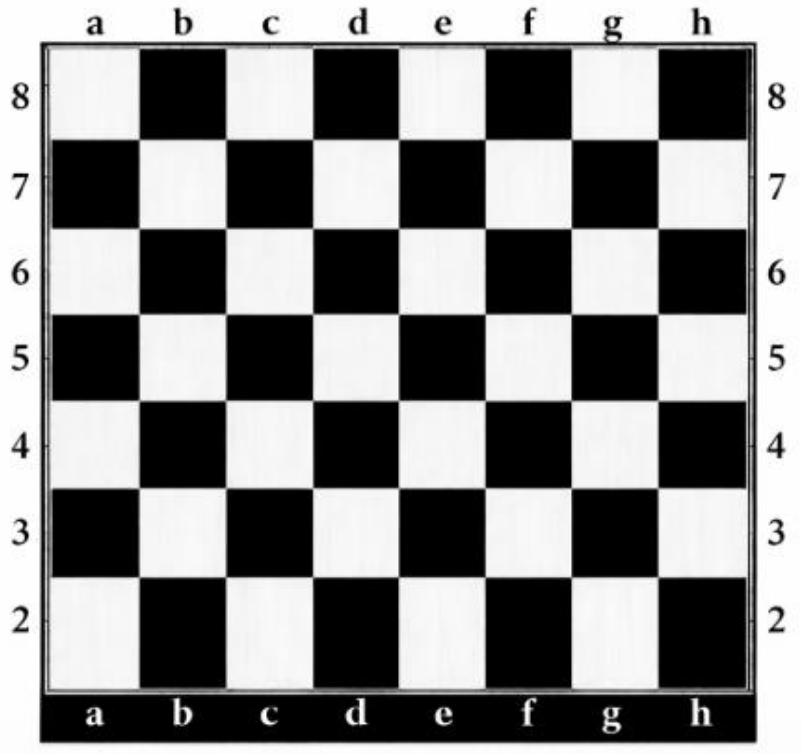
- *Movimientos de las Piezas*

Pieza	Movimiento
Rey	1 casilla en cualquier dirección (horizontal, vertical o diagonal). Enroque con torre.
Reina	N casillas en horizontal, vertical o diagonal.
Torre	N casillas en horizontal o vertical.
Alfil	N casillas en diagonal.
Caballo	Salta en "L": 2+1 casillas. Salta sobre otras piezas.
Peón	1 adelante (2 en primer movimiento). Captura 1 en diagonal. Promoción al llegar al final. Captura al paso.

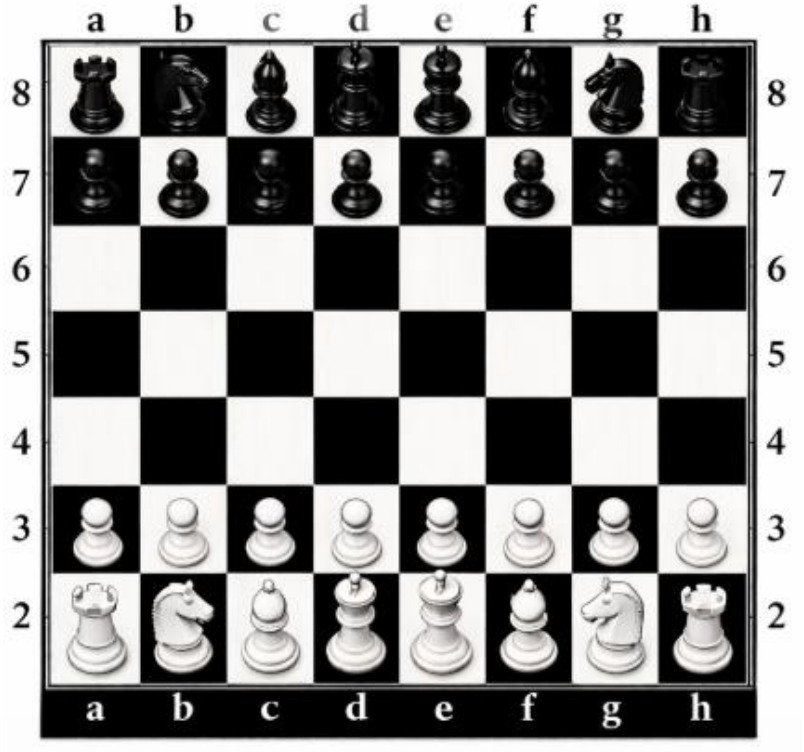
- *Referencia de movimientos*

Concepto	*Descripción breve*
Jaque	Ocurre cuando el rey está amenazado por una pieza enemiga. El jugador debe defenderse obligatoriamente en su siguiente turno.
Jaque mate	Ocurre cuando el rey está en jaque y no existe ningún movimiento válido para salvarlo. La partida termina y el jugador pierde.
Promoción	Ocurre cuando un peón llega a la última fila del tablero. El peón se transforma en otra pieza, generalmente una reina.

- *Referencia gráfica del tablero*



- *Referencia grafica del acomodo de piezas en el tablero*



- **Requerimientos funcionales a cumplir**

1. El proyecto debe ser desarrollado en el IDE Netbeans 26 y Java 24.
2. El proyecto debe ser llamado como Ajedrez_X, donde X representa el número de grupo.
3. El proyecto debe ser realizando utilizando una interfaz gráfica (no consola), la interfaz gráfica puede ser Java Swing o Java FX (preferiblemente).
4. Cuando se finalice el proyecto, deberá ser exportado como proyecto .zip
5. El proyecto debe permitir el juego entre los dos jugadores, validando el movimiento de piezas, los turnos, el historial de movimientos, el jaque, el jaque mate y la promoción y el determinar cual jugador gana la partida.
6. Sugerencia: Como parte del camino de solución de su proyecto, puede iniciar el modelo de su programación en modo consola y posteriormente, cuando sea validado el funcionamiento, cambiar a la interfaz gráfica.
7. Extras: Se reconocerán 5 puntos extra a la nota del proyecto, la implementación de la persistencia del proyecto mediante serialización y deserealización, es decir, poder almacenar la posición de las piezas en una partida y, posteriormente, el volver a colocarlas recuperando la posición y el turno del jugador correspondiente.

- **Aspectos Evaluativos**

La nota del proyecto se subdivide en dos partes:

Aspecto	Fecha	Valor
Defensa del Proyecto	14 de Mayo del 2026	15%
Implementación del Proyecto (Entrega)	12 de Mayo del 2026	85%

Se detalla como sigue:

- **Defensa del Proyecto:**

El proyecto será defendido por la totalidad de sus integrantes. La defensa consistirá en la revisión de una partida de ajedrez ejecutada sobre el sistema desarrollado, con el objetivo de verificar que todos los requerimientos funcionales hayan sido implementados correctamente. Adicionalmente, se realizarán consultas generales sobre el código fuente, por lo que los estudiantes deberán llevar impreso el diagrama UML correspondiente al diseño del sistema con fines de verificación y consulta.

- **Implementación del Proyecto (Entrega)**

1. La fecha de entrega máxima de este proyecto es el Martes 12 de Mayo del 2026 antes de las 11:59pm (GMT-6). **No se aceptarán proyectos posteriores a esa fecha-hora.**

2. El medio de entrega será por medio del Tec Digital, en donde se deberá subir un archivo comprimido (zipeado) con el siguiente nombre:

   IS2026_PP1_POO_Ajedrez_SUBGRUPO_X.zip
   Donde la X: Corresponde al número de cada subgrupo.

   En dicho archivo comprimido, se incluyen dos archivos:

   A) Un documento en PDF, con las siguientes secciones:
      a. Portada (Utilizar la que se encuentra al final de este documento)
      b. Diseño UML Utilizado (en Draw.io)

   B) Un archivo .zip que corresponde a su proyecto exportado de Netbeans con todo su código fuente, el nombre del proyecto será: **Ajedrez_X**
   Donde la X: Corresponde al número de cada subgrupo.

Otros Aspectos

1. Para dudas en cuanto al entendimiento del proyecto (**no en cuanto a diseño, implementación o corrección de errores**), el profesor las atenderá en un espacio posterior a la clase, por consulta o por Telegram.

2. **El uso de herramientas de inteligencia artificial para la codificación del proyecto está prohibido porque:**
   - Se busca evaluar las **habilidades reales de programación** del estudiante.
   - El objetivo del proyecto es que el estudiante demuestre su **comprensión de los conceptos de POO, lógica y diseño UML**.
   - El uso de IA puede generar código sin que exista un **aprendizaje significativo**.
   - Se requiere evidenciar el **proceso de análisis, diseño y desarrollo propio**.
   - Garantiza la **integridad académica** y evita el plagio automatizado.

Anexo 1. Portada estandarizada a utilizar en el trabajo:

Instituto Tecnológico de Costa Rica



Primer Proyecto Programado:
Implementación del juego de Ajedrez utilizando POO en Java

Grupo X:
Estudiante#1 – Carné: XXXXXXX
Estudiante#2 – Carné: XXXXXXX
Estudiante#3 – Carné: XXXXXXX

Profesor:
Ing. Emmanuel Ramírez Segura

Fecha de entrega:
DD/MM/AAAA

XX Semestre, 202X

Anexo 2. Rúbrica de la Defensa

Criterio	Ponderación	Excelente (100)	Bueno (75)	Regular (50)	Deficiente (25-0)	Total
Funcionamiento del juego durante la defensa	30%	La partida de prueba se desarrolla sin errores. Todos los movimientos son válidos, se respetan turnos, se detecta jaque, jaque mate y promoción.	La partida funciona correctamente en la mayoría de situaciones, con errores menores en movimientos poco comunes (enroque, captura al paso).	La partida presenta errores notorios (movimientos ilegales aceptados, fallos en detección de jaque o jaque mate) que afectan la demostración.	La partida no puede completarse por errores graves; el juego no respeta reglas básicas o no arranca correctamente.	
Errores encontrados durante la defensa	20%	No se presentan errores durante la ejecución de la partida de prueba. Todo funciona a la primera.	Se presentan errores menores no críticos (ej. retraso visual, historial mal actualizado) que no impiden continuar la partida.	Se presentan errores críticos (ej. la aplicación se cierra, un movimiento ilegal es aceptado, no detecta jaque mate) que impiden terminar la partida.	La aplicación falla repetidamente o se bloquea, haciendo imposible la demostración.	
Consultas sobre el código	20%	Todos los integrantes responden correctamente todas las preguntas sobre el código y su lógica.	La mayoría responde adecuadamente, con algunas dudas menores o respuestas parciales.	Varios integrantes no logran explicar partes del código que se les consulta.	Ninguno o casi ningún integrante puede responder las preguntas sobre el código.	
Diagrama UML impreso	10%	El UML es completo, claro y coherente con el código presentado.	El UML tiene la mayoría de los elementos, pero omite algún detalle menor.	El UML está incompleto o presenta incoherencias graves con el código.	El UML no se presenta, está totalmente vacío o no guarda relación con el proyecto.	
Participación de los integrantes	20%	Todos los integrantes participan activamente y demuestran conocimiento del proyecto.	Participan todos, pero con distribución desigual o algún integrante con participación muy limitada.	Solo uno o dos integrantes realizan la defensa; el resto apenas participa o no sabe qué responder.	Un solo integrante realiza toda la defensa; los demás no participan o están ausentes.	

Anexo 3. Rúbrica de la Implementación del Proyecto

Criterio	Ponderación	Excelente (100)	Bueno (75)	Regular (50)	Deficiente (25-0)	Total
Funcionalidad del ajedrez	55%	Implementa correctamente reglas: movimientos, capturas, turnos, jaque y jaque mate.	Funciona en su mayoría, con errores menores.	Funcionalidad parcial o con errores importantes.	No cumple con la lógica del ajedrez.	
Diseño orientado a objetos (POO)	25%	Uso correcto de abstracción, herencia y polimorfismo	Buen diseño con pequeños detalles.	Uso parcial o diseño poco claro.	No aplica correctamente POO.	
Interfaz (Swing/JavaFX)	10%	Interfaz funcional, clara y permite interactuar correctamente con el juego.	Interfaz funcional con detalles menores.	Interfaz limitada o poco clara.	No implementa interfaz funcional o la misma esta incompleta según lo que se solicitó (Turno, Jugador, Registro de Movimiento, entre otros).	
Calidad del código (buenas prácticas y nomenclatura)	5%	Código limpio, organizado, con nombres según las buenas prácticas en clases, atributos, metodos y constructores.	Presenta pocos errores menores.	Código poco claro o desordenado.	Código deficiente.	
Nombre del proyecto y archivos de entrega	5%	Cumple exactamente con nombres solicitados y estructura correcta.	Presenta un error menor.	Varios errores en nombres o estructura.	No cumple con lo solicitado.	